

Serving LLMs at Scale

Baseline vs vLLM

Workshop -- Hands-on Lab

Qwen2.5-1.5B-Instruct · Google Colab T4 · ~2-3 hrs

The Problem

You trained a model. You deploy it. Then:

Users	What happens
1	Works fine
10	Slow
50	Very slow
200	Timeout / crash

Why? -- Not the model. The **serving engine**.

Same weights. Same GPU. Different engine = 34x throughput difference.

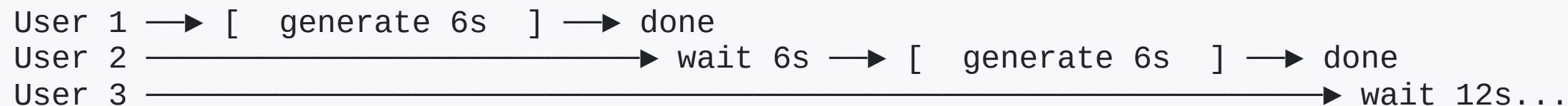
How Most People Serve LLMs

```
# The naive way -- what most tutorials show
output = model.generate(input, max_new_tokens=150)
```

- One request at a time
- Next request waits in queue
- GPU sits **idle** between requests
- Memory allocated statically -- most of it wasted

This is **HF Transformers baseline** -- correct output, terrible throughput.

Why It Breaks Under Load



- Sequential = throughput **flat** no matter how many users
- KV cache allocated as one contiguous block per request
→ wastes VRAM → fewer requests fit in memory
- No way to add a new request mid-generation

What is vLLM?

vLLM = inference engine built specifically for serving LLMs at scale.

Two core innovations:

1. PagedAttention

Manages KV cache like OS virtual memory -- non-contiguous blocks, no waste.

2. Continuous Batching

New requests join the running batch **immediately** -- no waiting for current batch to finish.

Same API as OpenAI. Drop-in replacement. No model changes needed.

PagedAttention -- Intuition

Before (static allocation):

[Request A: ] ← 50% wasted
[Request B: ] ← 62% wasted

After (PagedAttention):

[Block 1: A][Block 2: A][Block 3: B][Block 4: A][Block 5: B]...

- KV cache split into fixed-size **pages** (like OS memory pages)
- Pages assigned on demand, returned when request finishes
- No fragmentation → fit **2-3x more concurrent requests** in same VRAM

Continuous Batching -- Intuition

Static batching (old):

```
Batch [A, B, C] → wait for ALL to finish → next batch
                  ↑ C finished early but GPU idles
```

Continuous batching (vLLM):

```
[A, B, C] generating...
      C finishes → D joins immediately
[A, B, D] generating...
```

- Server always fully utilized
- Short requests don't block long ones
- Throughput scales **near-linearly** with concurrent users

The Numbers (T4 GPU · Qwen2.5-1.5B)

Exp A -- Single Request Latency

Engine	Latency
HF Baseline	6.47s
vLLM	2.47s

Exp B -- Concurrent Throughput

Concurrency	HF Baseline	vLLM	Ratio
1	24 tok/s	62 tok/s	2.6x
4	23 tok/s	229 tok/s	10x
16	23 tok/s	776 tok/s	34x

HF baseline is flat. vLLM scales. **That's the point.**

Workshop -- What You Will Do

~2-3 hours, 4 coding TODOs

#	TODO	Concept
1	Time <code>model.generate()</code> with <code>perf_counter</code>	GPU timing
2	Loop baseline over batch, accumulate tokens	Sequential serving
3	Call <code>client.chat.completions.create()</code>	OpenAI-compatible API
4	Fire all requests with <code>asyncio.gather()</code>	Async concurrency

Tools: Google Colab T4 · HF Transformers · vLLM · OpenAI Python SDK

Output: 4 log files + comparison chart saved to your Drive

Lab Structure

```
cell 1  -- GPU check
cell 2  -- Install (vllm, transformers, openai)
cell 3  -- Config & prompts

Section 1 -- HF Baseline
  cell 7  — TODO 1: baseline_generate()      ← timing
  cell 9  — TODO 2: Exp B loop                ← sequential serving

Section 2 -- vLLM
  cell 13 -- Start vLLM server (given)
  cell 15 — TODO 3: vllm_generate()          ← API call
  cell 17 — TODO 4: asyncio.gather()         ← concurrent requests

Section 3 -- Results
  cell 21 -- Chart: latency + throughput + speedup + accuracy
```

Want to Go Further?

Technique	Gain	Flag
Speculative decoding	2-3x latency	<code>--speculative-model</code>
Quantization (AWQ)	1.5-2x throughput	<code>--quantization awq</code>
Prefix caching	~0 cost for shared prompts	<code>--enable-prefix-caching</code>
Tensor parallelism	~Nx with N GPUs	<code>--tensor-parallel-size N</code>
Chunked prefill	lower tail latency	<code>--enable-chunked-prefill</code>

Full details → `docs/advance-technique.md`

Key Takeaways

1. **The model is not the bottleneck** -- the serving engine is
2. **PagedAttention** eliminates KV cache waste → more requests fit in VRAM
3. **Continuous batching** keeps GPU busy at all times → linear throughput scaling
4. **Same API** as OpenAI -- zero client code changes needed
5. **34x throughput** at concurrency 16 on a free Colab T4

"If the weights are identical, why does one handle 10 users and the other 200?"
-- You now know the answer.

Resources

- **vLLM docs:** <https://docs.vllm.ai>
- **PagedAttention paper:** Kwon et al., 2023 -- "Efficient Memory Management for Large Language Model Serving with PagedAttention"
- **Lab repo:** <https://github.com/Nukaze/vllm-concurrency-lab-101>
- **Solution notebook:** `docs/xd/lablab.ipynb`
- **Advanced techniques:** `docs/advance-technique.md`